

Mining Constraints for Event-based Monitoring in Systems of Systems

Thomas Krismayer*, Rick Rabiser†, and Paul Grünbacher‡

Christian Doppler Laboratory MEVSS

Institute for Software Systems Engineering

Johannes Kepler University Linz, Austria

*thomas.krismayer@jku.at, †rick.rabiser@jku.at, ‡paul.gruenbacher@jku.at

Abstract—The full behavior of software-intensive systems of systems (SoS) emerges during operation only. Runtime monitoring approaches have thus been proposed to detect deviations from the expected behavior. They commonly rely on temporal logic or domain-specific languages to formally define requirements, which are then checked by analyzing the stream of monitored events and event data. Some approaches also allow developers to generate constraints from declarative specifications of the expected behavior. However, independent of the approach, deep domain knowledge is required to specify the desired behavior. This knowledge is often not accessible in SoS environments with multiple development teams independently working on different, heterogeneous systems. In this New Ideas Paper we thus describe an approach that automatically mines constraints for runtime monitoring from event logs recorded in SoS. Our approach builds on ideas from specification mining, process mining, and machine learning to mine different types of constraints on event occurrence, event timing, and event data. The approach further presents the mined constraints to users in an existing constraint language and it ranks the constraints using different criteria. We demonstrate the feasibility of our approach by applying it to event logs from a real-world industrial SoS.

Index Terms—Constraint mining, event-based monitoring, systems of systems.

I. INTRODUCTION

Complex software-intensive systems are often described as systems of systems [17] comprising heterogeneous architectural elements. As the behavior of SoS fully emerges during operation only, runtime monitoring is needed to detect deviations from the requirements. Different research communities have been developing monitoring approaches for various kinds of systems and diverse types of checks [19], [25]: in many of these approaches the requirements are formally expressed using a form of temporal logic or as constraints in a domain-specific language (DSL). The requirements are then checked continuously at runtime by analyzing the monitored stream of events and event data, or post hoc by analyzing persisted event logs. For instance, in our earlier work we introduced REMINDS [27], a tool-supported monitoring approach for SoS. Our approach – similar to other approaches relying on the declarative specification of the expected behavior (e.g., [16]) – assumes that engineers manually define constraints [26] based on their domain knowledge. Due to the size, complexity, and heterogeneity of industrial SoS, however, the required knowledge is often only partly documented. Further, the continuous

and independent evolution of systems in the SoS requires to frequently define new and update existing constraints.

We are thus developing an approach to automatically extract constraint candidates from event logs recorded from different systems monitored in the SoS. Specifically, our approach uses rich event logs containing the type of events, their source in the SoS, and data associated with events (e.g., values from machine sensors or configuration parameters). Building on techniques from the field of specification mining [13], process mining [24], and machine learning [7] our approach aims at finding reappearing structures, patterns, and rules in these rich event logs to then suggest constraints to developers, who can easily include them in their monitoring infrastructure.

Specifically, we claim the following *contributions*: our approach (i) automatically mines both temporal constraints [3] and value constraints from rich event logs recorded from multiple systems in an SoS; (ii) it computes a set of constraint candidates, including a representation for users in a DSL [26]; and (iii) it ranks the constraint candidates using different criteria to ease their selection by users. This New Ideas Paper describes our constraint mining approach and presents a preliminary evaluation using event logs from a real-world SoS from the automation software domain. We outline the next steps in further developing and evaluating our approach.

II. BACKGROUND AND RELATED WORK

Event-based monitoring approaches [19] analyze the stream of events produced by a complex system to check its compliance to requirements or properties. The monitored events are commonly stored in event logs for later analyses. An event log consists of *event sequences*, each representing one run of a system. They contain multiple events of different types, which are typically sorted by their timestamps. An event sequence can be seen as an instance of a particular *event sequence type*, which has specific start and end event types. The information stored in event logs has implications on the types of constraints that can be mined (see, for example, the catalog by Autili et al. [2]):

Simple event logs only comprise the events together with their timestamps. They allow mining temporal constraints on the occurrence, order and timing of events [3]. In *rich event logs*, data elements of different types (e.g., String or Number) are additionally recorded for the events. Often the

data elements are further grouped hierarchically. This allows mining value constraints on specific data elements in the event log. For instance, the value of data elements may need to remain constant, it may need to lie above or below a specified threshold, or within a given range. The additional information in rich event logs can also help to distinguish if an event sequence can be regarded as valid or if unexpected behavior was monitored. This allows to use machine learning techniques [7] to mine additional data constraints based on classifiers trained using the event data from multiple valid and invalid event logs.

Existing specification mining techniques [13] aim at extracting (temporal) constraints from event logs and represent them as automata (e.g., [28]), linear temporal logic rules (e.g., [11]), or Markov chains (e.g., [1]). These techniques can be split into static algorithms (e.g., [8], [22]), which extract rules from analyzing source code, and dynamic specification mining algorithms (e.g., [5], [6], [10], [12], [18], [20]), which use method call traces or log files as input. Dynamic algorithms analyze traces based on available operations to find patterns. For example, this has been done in the context of Web pages [21]. Similarly, process mining approaches [24] discover temporal patterns in event traces (logs) to monitor and improve processes in different application domains. Dynamic analysis is also used to find value constraints for variables in programs [4]. Value constraints have also been extracted in the area of product line engineering using classifiers trained on multiple configurations of systems [23], [29].

Several existing approaches aim at discovering both temporal and value constraints: for instance, the approaches by Lo and Maoz [14] and Lorenzoli et al. [15] use dynamic analysis of method calls and values of program variables (e.g., method parameters) to mine constraints for program analysis and testing. Our approach also aims at mining temporal and value constraints, but additionally considers constraints on the timing of events. In contrast to existing specification mining techniques our approach processes events recorded with a monitoring tool. Further, after being presented and approved by users, the mined constraints are used for system monitoring.

III. RUNNING EXAMPLE

To illustrate and evaluate our approach we use events monitored from a real-world automation SoS from our industry partner Primetals Technologies, one of the world's leading engineering and plant-building companies in the metallurgical industry. Their plant automation SoS tracks and optimizes different stages of the metallurgical production process. Different, independently developed automation systems are distributed within a plant and size up to several million lines of code. The correct interplay between these software and hardware systems is crucial to guarantee continuous, uninterrupted production and high-quality products. Hence, it's essential to ensure that the SoS complies with its requirements. However, the full behavior of the SoS emerges only at runtime when the involved systems interact with each other, with the hardware, and with

third-party and legacy systems. Engineers thus need to capture events and data to monitor important requirements at runtime.

One system in our industry partner's SoS is the *Optimizer* supporting the cutting of steel slabs to minimize scrap and maximize the steel quality. Event logs for this system contain the following two event sequence types:

Sequence type 1 listed below represents a typical run of the Optimizer. It starts with a system requesting an optimization (A) and feeding data (B and C). The Optimizer then calculates the optimization result (D), feeds the result to the Cutting system, and notifies other systems in the SoS (E, F, and G).

```
A: Controller.requestOptimization
B: Optimizer.feedData_Start
C: Optimizer.feedData_Finished
D: Optimizer.calculateResult
E: Cutting.feedOptimizationResult
F: Tracking.updateSetpoints
G: Tracking.notifyListeners
```

Sequence type 2 represents a shorter version of the optimization run in which data is already available and the Optimizer starts (X), writes calculated optimization data to a database (Y), and then finishes (Z).

```
X: Optimizer.optimizeStart
Y: Optimizer.writeOptimizationData
Z: Optimizer.optimizeFinished
```

In the remainder of the paper we will use the symbols A–G and X–Z to denote these event names when explaining our approach. All these events have associated data elements, e.g., D is related with a data element containing the calculations the Optimizer performed on how to cut the steel slabs.

IV. APPROACH

Our approach aims at finding constraints based on structures and patterns [3] reappearing in rich event logs. It applies techniques from specification mining [13], process mining [24] and machine learning [7]. We build on REMINDS [27], which can record rich event logs in industrial SoS and uses a DSL to specify and check temporal and value constraints.

REMINDS records the origin of each event, i.e., the system in the SoS emitting it. Using domain knowledge, engineers can also define the start and end events of different system tasks. This allows us to split the events monitored from the SoS by system and event sequence, thus creating individual event logs for all runs of all systems in the SoS. As a result each event log used in our algorithm contains exactly one event sequence, i.e., the events produced by one sub-system during one run. Our constraint mining approach takes these (rich) event logs as input and automatically performs the following four steps also depicted in Figure 1.

A. Creating Sequence Type Trees

The first step aims at finding the most frequent event sequences for each event sequence type, following ideas from the field of process mining [24]. A tree with nodes containing

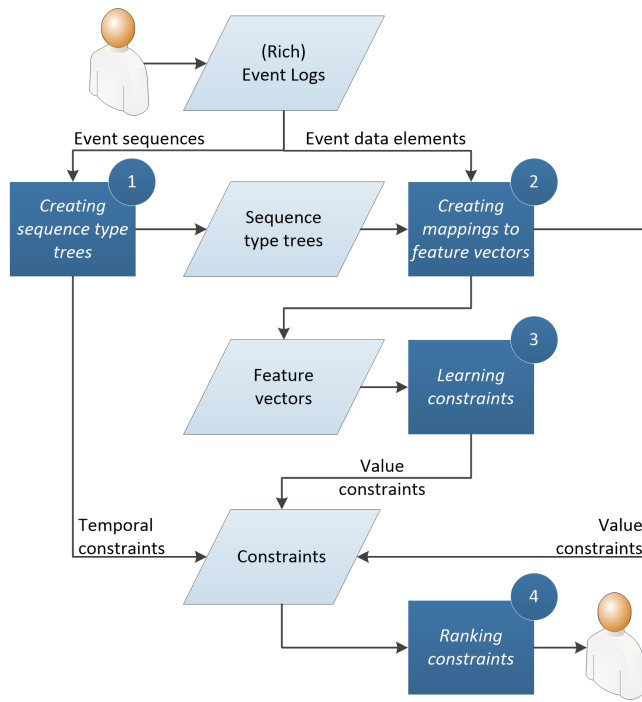


Figure 1. Four-step approach for mining and ranking constraint candidates.

event types is built for each event sequence type contained in the recorded event logs. The branches of the trees represent one specific order of events that appeared in at least one event sequence. Figure 2 shows an example of a sequence type tree created from the event types A to G (cf. Section III). There are six different branches, all starting and ending with the same events (A and E).

When processing one event sequence our approach increases the counter for each event in the respective branch or adds a new branch (with counter 1), if the tree does not yet cover the observed sequence. For example, adding another event sequence A-B-C-D-E to the tree shown in Figure 2, would increase the counter for these five events in the left-most branch by one – leading to 44 for A, 37 for B, and 34 for C, D, and E. However, when instead the sequence A-B-X-Y would be added, only the counters for A and B would be increased and a new branch for X-Y would be created as child of B – with the counter 1 for both X and Y.

After this first step our approach can already propose temporal constraints based on the events from the branch with the highest counter of each sequence type tree. One example from our experiments is the constraint ranked highest in Table II. It represents the branch with the highest count in Figure 2 (A-B-C-D-E). We thus assume that these events represent normal behavior, while the other branches are exceptions. However, even event sequences occurring only once could represent normal behavior. Our approach thus suggests different constraint candidates based on the frequency analysis.

Automatons built from extracting and merging behavioral patterns [6], [10], [22] lose information on the frequencies

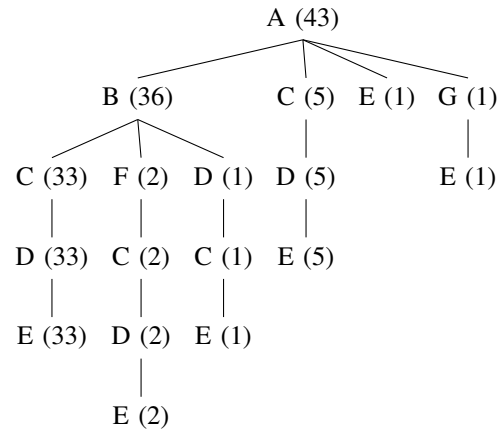


Figure 2. Example sequence type tree produced in step 1 of our approach (see Section III for the real names of the events from the industrial SoS).

of specific event sequence, thus making it impossible to determine the most common sequence needed for proposing temporal constraints. We thus use sequence trees as a simple way of distinguishing and counting different event sequences. A downside of these trees compared to automata is their higher use of memory. However, we only store the event types and counters in the tree to keep memory as low as possible.

B. Creating Mappings to Feature Vectors

In the second step our approach creates a mapping to a feature vector based on the sequence type trees. For this purpose we first extract the data elements (e.g., data element values, sources, and timestamps) for all event sequences in the most common branch of the sequence type tree, i.e., the tree branch with most sequences associated. Furthermore, our approach generates new data elements containing the time difference between the events based on their timestamps. This is done independently for each sequence type tree.

The approach then maps all event sequences from other branches containing the same order of events as the main sequence to a feature vector. However, we ignore data elements from any of these additional events. The reason for using sequences with additional events is that in an SoS multiple systems work in parallel, and events from other systems might occur at the same time. One example for this can be seen in Figure 2, where the branch A-B-C-D-E is most common. However, the algorithm also transforms the sequences A-B-F-C-D-E (containing an additional F) into feature vectors, but ignores the event F and the data elements associated with it.

During the mapping our algorithm aims to find data elements that contain the same value in all valid event logs. These *constant data elements* are assumed to remain unchanged for valid runs and are thus extracted as constraint candidates. For example, in Table I the data element `D.ExcOcc` (indicating that an exception occurred) is constant for all traces and is therefore extracted as a constraint specifying that `ExcOcc` has to be `false` for D. An example of a constraint extracted

Table I
EXAMPLE FIELDS OF FEATURE VECTORS PRODUCED IN STEP 2.

D.Thread	D.Process	D.ExcOcc	E.Process	time_0	time_1
T_243	TR_2_V14	false	TR_2_V14	36	169
T_239	TR_2_V14	false	TR_2_V14	84	35
T_243	TR_2_V14	false	TR_2_V14	49	35
T_239	TR_2_V14	false	TR_2_V14	52	113

in this step during our experiments is the second constraint shown in Table II.

Additionally, our algorithm detects constraints affecting multiple data elements from different events that need to have the same value in one event log (*equal data elements*). For instance, the event data element *D.Process* – containing the version of the software – equals the value of *E.Process* in all feature vectors (cf. Table I). It is therefore assumed that these two data elements will also share a value for all future logs and a constraint can be suggested to check this assumption. An example of a constraint extracted in this step during our experiments is the third constraint in Table II.

C. Learning Constraints

In the third step we use the machine learning tool suite Weka [9] to train a decision tree (using the J48 algorithm) and a rule-based classifier (using the JRip algorithm) from the feature vectors representing the event logs created in the second step. The model that was learned (i. e., the output from these classifiers) is analyzed and used to identify both valid (or invalid) event data element values for nominal attributes and thresholds for numeric values, which are then turned into constraints, similar to related techniques from the area of product line engineering [23], [29] as described in Section II.

The class for all vectors is set to *VALID* for event logs without defects and to *INVALID* for event logs containing defects (i. e., unexpected behavior). Event logs recorded by the monitoring tool could be marked as invalid by system users reporting unintended system behavior, after crashes, or by detecting unusually long delays in system execution.

For the rule-based classifiers we can parse the rules and transform all conditions that lead to a *VALID* classification as well as negations of conditions that lead to an *INVALID* classification into constraints.

We can also transform the branches of the decision trees into constraints that lead to a *VALID* classification. Branches of the decision tree that are classified as *INVALID* can be ignored, as their branching criterion is the negation of the path leading to a *VALID* leaf.

Both the rules and the branching criteria can be checks for a *constant data element* (e. g., $B.RunId = 1$) or for a *threshold* (e. g., $D.RunId > -1$). Additionally the rules can contain the basis for extracting *intervals* (cf. Section II), if one rule contains both an upper and a lower bound for a value, e. g., $(time_1 \geq 30)$ and $(time_1 \leq 169)$.

An example for a decision tree can be seen in Figure 3. In this part of the tree three different criteria need to be fulfilled to reach a leaf that is classified as *VALID*: $D.RunId > -1$,

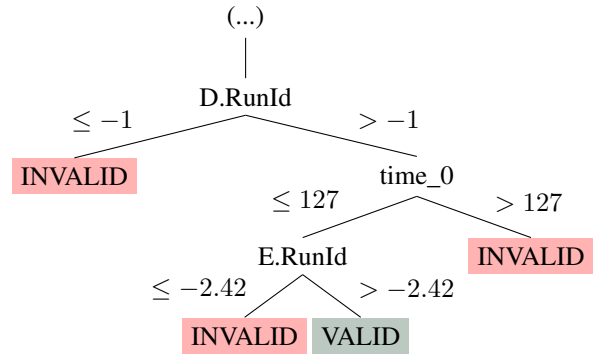


Figure 3. Part of a decision tree

$time_0 \leq 127$, and $E.RunId > -2.42$. These three conditions can be extracted as individual constraints. Two examples extracted in this step are the fourth and fifth constraints in Table II.

D. Ranking Constraints

In the final step the extracted constraints are ranked such that the constraints that are more likely to be accepted by users get a higher priority. The ranking is based on different scores for each constraint using the type of constraint and the number of the valid and invalid event logs for which the constraint is fulfilled for. Constraints satisfied for valid and violated by invalid event logs are given a higher score. Our algorithm also ranks temporal constraints higher than value constraints as temporal constraints directly reflect the actual, most common sequence of events representing system processes/machine cycles for the respective sequence type. They therefore have a high probability of being accepted as true constraints. However, this might not be applicable for other systems and might have to be changed, when adapting the algorithm for other scenarios. Furthermore, constraints that are fulfilled by less than a defined threshold percentage of the valid event logs (e. g., 50% in our case) are ignored.

E. Implementation

We have been implementing our four-step approach using Java and the Weka [9] tool API. Rich event logs are produced by REMINDS [27] as text files and event data is represented using JSON (<http://www.json.org>). Each step of our constraint mining approach is represented by an individual Java package: the earlier steps of the approach can be performed independently (e. g., if no class value is available as needed to perform step 3) and the individual steps can be easily exchanged or adapted. We currently produce a textual output of the ranked list of mined constraints.

V. PRELIMINARY EVALUATION

We conducted experiments using rich event logs from a real-world automation SoS of our industry partner (cf. Section III) to demonstrate the feasibility of our approach. We also collected feedback on the mined and ranked constraints from experienced engineers of our industry partner.

Table II
EXAMPLE CONSTRAINTS MINED AND RANKED BY OUR APPROACH BASED ON RICH EVENT LOGS FROM THE INDUSTRIAL SoS.

Rank	Constraint	Scores
1	if event "Controller.requestOptimization" occurs events "Optimizer.feedData_Start", "Optimizer.feedData_Finished", "Optimizer.calculateResult", "Cutting.feedOptimizationResult" occur consecutively	Temporal Constraint [35/43; 35/43]
2	if event "Controller.requestOptimization" occurs data ("General", "Process") == "TrackingCore"	Value Constraint [43/43; 41/43]
3	if event "Controller.requestOptimization" occurs data ("General", "Process") == "Cutting.feedOptimizationResult", ("General", "Process")	Value Constraint [43/43; 41/43]
4	if event "Optimizer.feedData_Finished" occurs data ("General", "Process") == "TrackingCore"	Value Constraint [41/41; 36/41]
5	if event "Optimizer.feedData_Start" occurs time until event "Optimizer.feedData_Finished" <= 0.271 seconds	Value Constraint [36/36; 32/36]

A. Experimental Setup

We recorded the event logs using a simulator provided by our industry partner that executes the actual steel casting automation software but simulates the plant hardware based on real plant data recorded earlier. We instrumented the SoS with probes created by engineers of our industry partner using REMINDS.

The event logs used for our experiments contained a total of 70 sequences of the two different sequence types (A-G and X-Z) described in Section III. To test our approach we modified the sequences in the rich event logs by automatically seeding random differences, i. e., for each of the event sequences both the original version and a second randomly altered version are used – resulting in a total of 140 sequences. Our modifications included changing the value of individual data elements and changing the time between two events. For each event log our algorithm randomly selected and applied one of these change types to create the altered version. Event sequences with differences were considered as INVALID for the purpose of this evaluation, while event sequences without changes were considered as VALID.

B. Results

We describe the results of our preliminary evaluation following the four steps of our approach:

Creating Sequence Type Trees. Step 1 resulted in two sequence type trees, one of which is depicted in Figure 2. Due to the cyclic behavior of the Optimizer system in both trees one branch clearly represents the most common event sequence – 33 of 43 and 26 of 27 event logs, respectively – allowing to suggest the first constraint for each sequence type.

Mapping to Feature Vectors. In the second step our approach mapped the event logs that fit to the most common branch of each sequence type tree to feature vectors. The resulting feature vectors include the event data extracted from the event logs and the time between the events. Examples of partial feature vectors can be seen in Table I. In this step a total of 28 value constraints could be mined and suggested.

Learning Constraints. The third step created a decision tree (using J48) and a rule set (using JRip) for each event sequence type and analyzed these two prediction models. From these analyses our approach could learn and suggest a total of 13 constraints for the two sequence types.

Ranking Constraints. In the last step our algorithm ranked all yielded constraints by type and frequency of violation. Table II shows examples of constraints and the scores calculated to rank them (cf. Section IV-D).

Overall, 43 constraints were automatically mined using our approach – 26 for the first sequence and 17 for the second sequence. We showed the 15 highest ranked constraints of the more common sequence to engineers from our industry partner and they regarded 11 of these 15 mined constraints as useful for inclusion in the monitoring infrastructure. Considering that our approach works fully automatic and considering the limited size of the event logs we used in our preliminary evaluation these results are promising, indicating the feasibility of our approach. Further, as confirmed by our industry partner, even mining only a few constraints automatically is a huge benefit compared to manual definition of constraints.

C. Threats to Validity

Our preliminary results demonstrate the basic feasibility of our approach. Even though we used rather small event logs from just one system and randomly modified these logs to produce invalid logs, we regard them as representative for larger logs we have seen so far in the SoS of our industry partner with regard to cyclic behavior and the structure of event data. Regarding the scalability of our approach we yet have to conduct larger experiments. We further have to apply our approach to other systems to investigate its generalizability.

A threat regarding the learning part (step 3) of our approach is that when using our approach in practice it might not always be possible to obtain rich event logs that contain a class label valid vs. invalid, e. g., if no runs with unintended behavior occurred or this is not known. However, there are several possible ways to deal with this issue: one solution is to mine constraints based on the distribution of the observed values, e. g., define an interval based on the lowest and highest value encountered or define a set of valid nominal values including the most commonly observed value(s). Our approach, however, is also useful even when skipping the learning part altogether and only using the constraints extracted in the first two steps.

VI. CONCLUSION AND FUTURE WORK

We presented an approach for automatically mining constraints from (rich) event logs recorded by a monitoring tool in an SoS. Our approach builds on ideas from specification

mining, process mining, and machine learning. It combines and enhances existing algorithms to use them in the field of event-based monitoring. While our approach uses rich event logs to mine candidates for temporal and value constraints it can also mine certain constraints based on simple event logs. Our algorithm represents an automated solution for a real-world industry problem: definition and maintenance of constraints. Using examples from a real-world industrial automation SoS, we demonstrated that our approach is capable of extracting constraints expressed in an existing DSL. 11 of 15 mined constraints were regarded as useful by engineers of our industry partner.

In our future work we will apply our approach to other systems and conduct experiments with larger event logs and different systems to test its generalizability. Furthermore, we will implement end-user tools allowing engineers to configure our approach and to select the ranked constraint candidates and automatically activate them in a monitoring tool. Also, we plan to extend our approach allowing users to provide additional information – e. g., domain-knowledge about known cycles and issues – to the mining and learning algorithms to mine more constraints and even other types of constraints.

ACKNOWLEDGMENTS

This work has been supported by the Christian Doppler Forschungsgesellschaft Austria and Primetals Technologies.

REFERENCES

- [1] E. Alevizos, A. Artikis, and G. Paliouras. Event Forecasting with Pattern Markov Chains. In *Proc. of the 11th ACM Int'l Conf. on Distributed and Event-based Systems*, pages 146–157. ACM, 2017.
- [2] M. Autili, L. Grunske, M. Lumpe, P. Pelliccione, and A. Tang. Aligning Qualitative, Real-Time, and Probabilistic Property Specification Patterns Using a Structured English Grammar. *IEEE Trans. Soft. Eng.*, 41(7):620–638, 2015.
- [3] M. Dwyer, G. Avrunin, and J. Corbett. Patterns in property specifications for finite-state verification. In *Proc. of the 1999 Int'l Conf. on Software Engineering*, pages 411–420. IEEE, 1999.
- [4] M. Ernst, J. Perkins, P. Guo, S. McCamant, C. Pacheco, M. Tschantz, and C. Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1):35–45, 2007.
- [5] D. Fahland, D. Lo, and S. Maoz. Mining branching-time scenarios. In *Proc. of the 28th IEEE/ACM Int'l Conf. on Automated Software Engineering*, pages 443–453, 2013.
- [6] M. Gabel and Z. Su. Javert: fully automatic mining of general temporal properties from dynamic traces. In *Proc. of the 16th ACM SIGSOFT Int'l Symp. on Foundations of Software Engineering*, pages 339–349. ACM, 2008.
- [7] D. Goldberg and J. Holland. Genetic algorithms and machine learning. *Machine Learning*, 3(2):95–99, 1988.
- [8] N. Gruska, A. Wasylkowski, and A. Zeller. Learning from 6,000 Projects: Lightweight Cross-project Anomaly Detection. In *Proc. of the 19th Int'l Symp. on Software Testing and Analysis*, pages 119–130. ACM, 2010.
- [9] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. Witten. The WEKA Data Mining Software: An Update. *SIGKDD Explorations*, 11(1):10–18, 2009.
- [10] T.-D. Le, X.-B. Le, D. Lo, and I. Beschastnikh. Synergizing Specification Miners Through Model Fissions and Fusions (T). In *Proc. of the 30th IEEE/ACM Int'l Conf. on Automated Software Engineering*, pages 115–125. IEEE, 2015.
- [11] C. Lemieux, D. Park, and I. Beschastnikh. General LTL Specification Mining (T). In *Proc. of the 30th IEEE/ACM Int'l Conf. on Automated Software Engineering*, pages 81–92. IEEE, 2015.
- [12] W. Li, A. Forin, and S.A. Seshia. Scalable specification mining for verification and diagnosis. In *Proc. of the 47th Design Automation Conf.*, pages 755–760. ACM, 2010.
- [13] D. Lo, S.-C. Khoo, J. Han, and C. Liu. *Mining Software Specifications: Methodologies and Applications*. CRC Press, 2011.
- [14] D. Lo and S. Maoz. Scenario-based and value-based specification mining: better together. *Automated Software Engineering*, 19(4):423–458, 2012.
- [15] D. Lorenzoli, L. Mariani, and M. Pezzè. Automatic Generation of Software Behavioral Models. In *Proc. of the 30th Int'l Conf. on Software Engineering*, pages 501–510. ACM, 2008.
- [16] M. Montali, M. Pesic, W. van der Aalst, F. Chesani, P. Mello, and S. Storari. Declarative Specification and Verification of Service Choreographies. *ACM Trans. Web*, 4(1):3:1–3:62, January 2010.
- [17] C. Nielsen, P. Larsen, J. Fitzgerald, J. Woodcock, and J. Peleska. Systems of Systems Engineering: Basic Concepts, Model-Based Techniques, and Research Directions. *ACM Comput. Surv.*, 48(2):18:1–18:41, 2015.
- [18] M. Pradel and T. Gross. Automatic Generation of Object Usage Specifications from Large Method Traces. In *Proc. of the 24th IEEE/ACM Int'l Conf. on Automated Software Engineering*, pages 371–382. IEEE, 2009.
- [19] R. Rabiser, S. Guinea, M. Vierhauser, L. Baresi, and P. Grünbacher. A Comparison Framework for Runtime Monitoring Approaches. *Journal of Systems and Software*, 125(March):309–321, 2017.
- [20] G. Reger, H. Barringer, and D. Rydeheard. A pattern-based approach to parametric specification mining. In *Proc. of the 28th IEEE/ACM Int'l Conf. on Automated Software Engineering*, pages 658–663. IEEE, 2013.
- [21] M. Schur, A. Roth, and A. Zeller. Mining Behavior Models from Enterprise Web Applications. In *Proc. of the 9th Joint Meeting on Foundations of Software Engineering*, pages 422–432. ACM, 2013.
- [22] S. Shoham, E. Yahav, S.J. Fink, and M. Pistoi. Static Specification Mining Using Automata-Based Abstractions. *IEEE Trans. Soft. Eng.*, 34(5):651–666, 2008.
- [23] P. Temple, J.A. Galindo, M. Acher, and J.-M. Jézéquel. Using Machine Learning to Infer Constraints for Product Lines. In *Proc. of the 20th Int'l Systems and Software Product Line Conf.*, pages 209–218. ACM, 2016.
- [24] W. van der Aalst, A. Adriansyah, A.K.A. Medeiros, F. Arcieri, T. Baier, T. Blickle, J.C. Bose, P. Brand, R. Brandtjen, J. Buijs, et al. Process mining manifesto. In *Business Process Management Workshops*, pages 169–194. Springer, 2012.
- [25] M. Vierhauser, R. Rabiser, and P. Grünbacher. Requirements Monitoring Frameworks: A Systematic Review. *Information and Software Technology*, 80(December):89–109, 2016.
- [26] M. Vierhauser, R. Rabiser, P. Grünbacher, and A. Egyed. Developing a DSL-Based Approach for Event-Based Monitoring of Systems of Systems: Experiences and Lessons Learned. In *Proc. of the 30th IEEE/ACM Int'l Conf. on Automated Software Engineering*, pages 715–725. IEEE, 2015.
- [27] M. Vierhauser, R. Rabiser, P. Grünbacher, K. Seyerlehner, S. Wallner, and H. Zeisel. ReMinds: A Flexible Runtime Monitoring Framework for Systems of Systems. *Journal of Systems and Software*, 112:123–136, 2016.
- [28] W. Weimer and G.C. Necula. Mining Temporal Specifications for Error Detection. In *Proc. of the 11th Int'l Conf. on Tools and Algorithms for the Construction and Analysis of Systems*, pages 461–476. Springer, 2005.
- [29] L. Yi, W. Zhang, H. Zhao, Z. Jin, and H. Mei. Mining binary constraints in the construction of feature models. In *Proc. of the 20th IEEE Int'l Requirements Engineering Conf.*, pages 141–150. IEEE, 2012.